

Git for Developers

Module 11 — Integration & Ecosystem

IDEs, GUI Tools, CI/CD Pipelines, and Project Management

Module 11 — Agenda

Git in Your Tools

- Git with IDEs: VS Code, IntelliJ / JetBrains
- GUI clients: GitKraken, Sourcetree, GitHub Desktop

Git in Your Workflow

- Git and CI/CD pipelines: GitHub Actions, GitLab CI
- Connecting Git to project management: Jira, Linear, Trello

Part 1 — Git with IDEs

Git in VS Code

VS Code has first-class Git integration via the **Source Control panel** (`Ctrl+Shift+G`):

Feature	How to access
Stage files	Click <code>+</code> next to a file in Source Control
Unstage	Click <code>-</code> next to staged file
Commit	Enter message + <code>Ctrl+Enter</code>
Push / Pull	Click the sync button in the status bar
View diffs	Click any modified file

Useful VS Code Git Extensions

Extension	What it adds
GitLens	Rich blame, history, comparison, and visualization
Git Graph	Visual branch graph in the IDE
Git History	Browse file and line history interactively
GitHub Pull Requests	Manage PRs directly from VS Code
Conventional Commits	Guided commit message writing

Git in IntelliJ / JetBrains IDEs

All JetBrains IDEs (IntelliJ IDEA, PyCharm, GoLand, WebStorm) share the same Git UI:

- **Git tool window** (`Alt+9` or `View → Tool Windows → Git`) — branch graph, log
- **Commit tool window** — stage, diff, and commit with amend support
- **Branches popup** (`` Ctrl+ ` ``) — create, switch, merge, rebase branches
- **Inline blame** — shows last commit per line in the editor
- **Revert / Annotate / Show History** — right-click any file or selection

IntelliJ's **interactive rebase** UI is one of the best visual rebase tools

Part 2 — GUI Tools

GUI Git Clients

Tool	Platform	Best for
GitHub Desktop	macOS, Windows	Beginners, GitHub workflows
Sourcetree	macOS, Windows	Gitflow support built in
GitKraken	macOS, Windows, Linux	Visual power users, built-in Jira/GitHub/GitLab
Fork	macOS, Windows	Fast, clean UI, interactive rebase

When to Use a GUI Client

GUIs shine for:

- **Visual branch graphs** — seeing your entire history at a glance
- **Interactive rebase** — drag and drop commits to reorder/squash
- **Partial staging** — selecting specific lines to stage in a visual diff
- **Conflict resolution** — 3-panel merge view
- **Onboarding new developers** — lower barrier to entry

The command line is faster for simple operations. GUIs are better for complex history manipulation and visual exploration.

Part 3 — Git & CI/CD Pipelines

Git as the Source of Truth for Deployments

Git drives the entire CI/CD pipeline:

```
git push → trigger CI → tests pass → build artifact → deploy
      |
git tag v2.1.0 → trigger CD → deploy to production
      |
git push hotfix/ → trigger CI → fast path to staging
```

The **branch** or **tag** that triggered the pipeline determines where the artifact is deployed.

GitHub Actions – Basics

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, 'feature/**']
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```

GitHub Actions – Deploy on Tag

```
name: Release

on:
  push:
    tags:
      - 'v*'      # trigger on any tag like v1.0.0, v2.3.1

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

GitLab CI — Basics

```
# .gitlab-ci.yml
```

```
stages:
```

- test
- build
- deploy

```
test:
```

```
  stage: test
```

```
  image: node:20
```

```
  script:
```

- npm ci
- npm test

```
only:
```

Branch-Based Deployment Strategy

Branch / Tag	Environment	Trigger
<code>feature/*</code>	—	CI: tests only
<code>main</code>	Staging	CI + auto-deploy to staging
<code>release/*</code>	UAT	CI + deploy to UAT on PR merge
<code>v*</code> (tag)	Production	CD: deploy to production
<code>hotfix/*</code>	Staging → Production	Fast-track pipeline

Part 4 — Git & Project Management

Linking Git to Issue Trackers

Most platforms automatically link commits to issues via keywords in commit messages:

```
# GitHub – close an issue with a commit or PR
```

```
git commit -m "fix: prevent login bypass"
```

```
Closes #142
```

```
Fixes #143"
```

```
# Jira – link a commit to a Jira ticket
```

```
git commit -m "PROJ-142: fix token expiry validation"
```

```
# GitLab – close an issue
```

Jira + GitHub / GitLab Integration

Branch naming convention for Jira

```
# Include the Jira ticket in the branch name  
git checkout -b PROJ-142-fix-token-expiry
```

- Jira auto-links the branch to the ticket when the name matches
`PROJ-XXX`
- Commits, PRs, and build status appear on the Jira issue
- Transitioning the Jira issue can be automated via commit message keywords

Smart commits

Git + Slack / Teams Notifications

```
# GitHub Actions – notify Slack on deploy
- name: Notify Slack
  uses: slackapi/slack-github-action@v1
  with:
    channel-id: '#deployments'
    slack-message: |
      *Deployed ${github.ref_name}* to production
      Triggered by: ${github.actor}
      Commit: ${github.sha}
  env:
    SLACK_BOT_TOKEN: ${secrets.SLACK_BOT_TOKEN}
```

Summary — Integration & Ecosystem

Area	Key tools / practices
VS Code	Built-in Git + GitLens, Git Graph extensions
JetBrains	Native Git UI, interactive rebase, inline blame
GUI clients	GitKraken, Sourcetree, Fork — visual branch management
GitHub Actions	YAML workflows triggered by push/PR/tag
GitLab CI	<code>.gitlab-ci.yml</code> — built-in CI/CD on every repo